

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**



ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И УПРАВЛЯЮЩИХ СИСТЕМ

Лабораторная работа №1
по дисциплине: **Операционные системы**
тема: **«Базовая работа с процессами в ОС Linux. Сигналы»**

Выполнил: ст. группы ВТ-231
Ковалев А. В.

Проверили:
Островский А. М.

Белгород 2025 г

Лабораторная работа № 1

Цель работы: Изучить основы работы с процессами в ОС Linux, а также освоить сигналы как

универсальное средство межпроцессного взаимодействия.

Цель работы обуславливает постановку и решение следующих **задач**:

1. Изучить основные понятия о процессах в ОС Linux, относящиеся к их созданию и управлению.
2. Ознакомиться с базовыми системными вызовами POSIX для работы с процессами: `fork()`, `wait()`, `waitpid()`.
3. Изучить назначение и возможности сигналов в Linux, их классификацию (системные, пользовательские) и обработку.
4. Освоить установку обработчиков сигналов с помощью `signal()` и `sigaction()`, а также способы отправки сигналов с помощью `kill()` и `sigqueue()`.
5. Познакомиться с соответствующими консольными инструментами для мониторинга и управления процессами и сигналами.
 - 5.1 Изучение команд `ps`, `top`, `htop` — просмотр списка процессов, их состояний и ресурсов.
 - 5.2 Изучение команд `jobs`, `bg`, `fg` — управление фоновыми и приостановленными задачами в оболочке.
 - 5.3 Изучение команд `kill`, `pkill`, `killall` — отправка сигналов процессам по PID или имени.
 - 5.4 Изучение команды `ulimit` — просмотр и установка ресурсных ограничений (например, число процессов, размер стека).
 - 5.5 Изучение команды `strace` — отслеживание системных вызовов процесса, включая работу с сигналами.
6. Выполнить индивидуальное задание для закрепления знаний на практике, подготовив соответствующую программу на языке C (номер задания соответствует номеру студента по журналу; если этот номер больше, чем максимальное число заданий, тогда вариант задания вычисляется по формуле: номер по журналу % максимальный номер задания, где % — остаток от деления; если остаток равен 0, студенту назначается задание с максимальным номером).
7. Подготовить отчёт по работе, который должен включать: краткое описание всех использованных системных вызовов и команд; текст программы; протоколы; скриншоты с демонстрацией поведения ОС Linux; выводы.

Вариант 9

Реализовать «пинг-понг» сигналов между родительским процессом и дочерним процессом. Родитель первым генерирует целое число X и отправляет его дочернему процессу («пинг»). Дочерний процесс, получив сигнал, печатает на экран X с указанием своего PID и отправляет родителю новое целое число X' («понг»). Родитель, получив X' , печатает его со своим PID, генерирует новое целое число X'' и снова отправляет дочернему процессу. Таким образом процессы обмениваются сигналами в бесконечном цикле. Корректное завершение осуществляется по `Ctrl+C`: родитель перехватывает `SIGINT`, печатает сообщение об остановке и посылает дочернему процессу `SIGTERM`.

Выполнение индивидуального задания:

Исходный код программы:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <time.h>

static volatile pid_t child_pid = 0;
static volatile sig_atomic_t stop_flag = 0;

// Обработчик в родителе: получает "понг" от дочернего процесса
void parent_handler(int signo, siginfo_t *si, void *context) {
    int val = si->si_value.sival_int;
    printf("[Parent %d] получил число: %d\n", (int)getpid(), val);

    sleep(1); // Пауза для наглядности

    int new_val = rand() % 100;
    printf("[Parent %d] отправляет число: %d\n", (int)getpid(), new_val);

    union sigval sv;
    sv.sival_int = new_val;
    sigqueue(child_pid, SIGUSR1, sv);
}

// Обработчик в дочернем процессе: получает "пинг" от родителя
void child_handler(int signo, siginfo_t *si, void *context) {
    int val = si->si_value.sival_int;
    printf("[Child %d] получил число: %d\n", (int)getpid(), val);

    sleep(1); // Пауза для наглядности

    int new_val = rand() % 100;
    printf("[Child %d] отправляет число: %d\n", (int)getpid(), new_val);

    union sigval sv;
    sv.sival_int = new_val;
    sigqueue(getppid(), SIGUSR2, sv);
}

// Обработчик SIGINT (Ctrl+C) — только для родителя
void stop_handler(int signo) {
    printf("\n[Parent %d] Получен SIGINT. Завершение...\n", (int)getpid());
```

```

    if (child_pid > 0) {
        kill(child_pid, SIGTERM);
    }
    stop_flag = 1;
}

int main() {
    setvbuf(stdout, NULL, _IONBF, 0); // Отключаем буферизацию
    srand(time(NULL));

    // Устанавливаем обработчик для "понга" (SIGUSR2)
    struct sigaction sa;
    sa.sa_sigaction = parent_handler;
    sa.sa_flags = SA_SIGINFO;
    sigemptyset(&sa.sa_mask);
    sigaction(SIGUSR2, &sa, NULL);

    // Обработчик завершения
    signal(SIGINT, stop_handler);

    child_pid = fork();
    if (child_pid == -1) {
        perror("fork");
        exit(1);
    }

    if (child_pid == 0) {
        // Дочерний процесс
        signal(SIGINT, SIG_DFL); // Восстанавливаем стандартное поведение

        struct sigaction sa_ch;
        sa_ch.sa_sigaction = child_handler;
        sa_ch.sa_flags = SA_SIGINFO;
        sigemptyset(&sa_ch.sa_mask);
        sigaction(SIGUSR1, &sa_ch, NULL);

        while (1) {
            pause(); // Ждём сигнал
        }
    } else {
        // Родительский процесс
        sleep(1); // Даем ребёнку время установить обработчик

        int first = rand() % 100;
        printf("[Parent %d] отправляет число: %d\n", (int)getpid(), first);
    }
}

```

```

union sigval sv;
sv.sival_int = first;
sigqueue(child_pid, SIGUSR1, sv);

while (!stop_flag) {
    pause(); // Ждём сигнала от ребёнка
}

wait(NULL);
printf("[Parent %d] Дочерний процесс завершён. Выход.\n", (int)getpid());
}

return 0;
}

```

Результат выполнения программы:

```

~/projects/university/5sem/os/lab1 master* > ./res
[Parent 28636] отправляет число: 97
[Child 28637] получил число: 97
[Child 28637] отправляет число: 97
[Parent 28636] получил число: 97
[Parent 28636] отправляет число: 7
[Child 28637] получил число: 7
[Child 28637] отправляет число: 7

```

Результат выполнения команды `pgrep -l res`

```

28636 res
28637 res

```

Вывод:

В ходе выполнения лабораторной работы №1 были изучены основы работы с процессами в ОС Linux, включая их создание с помощью `fork()`, синхронизацию и завершение через `wait()`. Освоены механизмы обработки сигналов с использованием `signal()` и `sigaction()`, а также отправка сигналов через `kill()` и `sigqueue()`. Практически реализовано межпроцессное взаимодействие по принципу «пинг-понг» (задание №9), продемонстрирована корректная обработка завершения по `SIGINT` и управление дочерними процессами. С помощью консольных утилит подтверждена работоспособность программы и изучено поведение процессов в реальном времени. Лабораторная работа позволила закрепить ключевые концепции управления процессами и сигналами в Linux.